



ITM SKILLS
UNIVERSITY

FINAL PROJECT REPORT

Bachelor of Technology in Computer Science and Engineering
Class BTech CSE 25–29

PROJECT TITLE

Parking Fee Calculator

Submitted By	Rashmeet Kaur Nanade
---------------------	----------------------

Roll No:	150096725165
-----------------	--------------

Cohort:	Demis Hassabis
----------------	----------------

Subject	Java Programming
----------------	------------------

1. Project overview

This project is a Java application that calculates the parking charge for a vehicle in a commercial parking facility. Instead of working out the fee by hand, the user enters the vehicle number, selects the vehicle type and types the entry and exit time. The program works out how long the vehicle was parked, applies the rate for that vehicle type and the correct duration slab, and prints a parking receipt.

The program can be run in two ways. Console mode uses the Scanner class and is the main implementation required by the case study. GUI mode is an extra feature built with Java Swing, which gives the same calculation in a window with text fields, a drop-down list and a receipt area. The Java topics used are Scanner, switch-case, operators, if-else statements and methods.

2. Problem statement

A commercial parking facility requires a system to calculate parking charges based on vehicle type and parking duration. The system should accept vehicle information, calculate the parking duration, apply vehicle-specific rates and generate a parking receipt.

The rates used in this project are Rs. 20 per hour for a bike, Rs. 40 per hour for a car and Rs. 60 per hour for a truck. A surcharge is added for longer parking, as explained in Section 5.4.

3. Objectives

- Accept vehicle information from the user.
- Calculate the parking duration from the entry and exit time.
- Apply the hourly rate that belongs to the vehicle type.
- Apply different duration slabs to the fee.
- Generate a parking receipt.
- Practise Scanner, switch-case, operators, if-else statements and methods in a working program.
- Offer an optional graphical interface along with the console version.

4. Scope of the project

The project calculates the fee for one vehicle at a time. It supports three vehicle types (Bike, Car and Truck) and takes the times in 24-hour HH:mm format. A vehicle that enters before midnight and leaves after midnight is handled, so the duration is correct for parking that crosses midnight.

It is meant to show how the fundamental Java concepts fit together, so it is not a complete parking management system. It does not store any vehicle records, does not track dates (only the time of day) and has no database or payment feature. Parking for more than 24 hours cannot be calculated, because only the time of day is entered.

5. System modules

The program is divided into the five modules given in the case study. The calculations are done by separate methods, and the console and GUI versions both reuse them.

5.1 Vehicle entry

The user enters the vehicle number. In console mode it is read with `scanner.nextLine()` and in GUI mode from a text field. The number is trimmed and the program refuses an empty value.

5.2 Vehicle type selection

In console mode the user chooses 1 for Bike, 2 for Car or 3 for Truck, and a switch statement converts the choice into the vehicle type. In the GUI the user picks the type from a drop-down list. Any other choice is rejected as an invalid vehicle choice.

5.3 Duration entry

The user enters the entry time and the exit time in HH:mm format. The method `calculateDuration()` converts both strings to `LocalTime` values and finds the difference in minutes with `Duration.between()`. If the difference is negative, 24×60 minutes are added, which handles parking after midnight. If either time cannot be read, the method returns -1 and the program shows an error. The method `getParkingHours()` then rounds the minutes up to whole hours with `Math.ceil()`, so any part of an hour is charged as a full hour.

5.4 Fee calculation

The method `calculateFee()` uses a switch statement to pick the hourly rate for the vehicle type. An if-else ladder then applies the duration slab:

Parking hours	Surcharge	Fee formula
Up to 2 hours	None	$\text{rate} \times \text{hours}$
3 to 5 hours	20%	$\text{rate} \times \text{hours} + (\text{rate} \times \text{hours} \times 0.20)$
More than 5 hours	40%	$\text{rate} \times \text{hours} + (\text{rate} \times \text{hours} \times 0.40)$

Vehicle type	Hourly rate	Switch case
Bike	Rs. 20	case "Bike"
Car	Rs. 40	case "Car"
Truck	Rs. 60	case "Truck"

For example, a car parked from 10:00 to 14:00 is parked for 4 hours. The base fee is $40 \times 4 = 160$, and because 4 hours falls in the 3 to 5 hour slab, a 20% surcharge of 32 is added. The total fee is Rs. 192.00.

The table below gives the complete fee chart produced by these rules (in Rs.), which is also useful for checking the program output.

Hours	Slab	Surcharge	Bike	Car	Truck
1	Up to 2	None	20.00	40.00	60.00
2	Up to 2	None	40.00	80.00	120.00
3	3 to 5	20%	72.00	144.00	216.00
4	3 to 5	20%	96.00	192.00	288.00
5	3 to 5	20%	120.00	240.00	360.00
6	Above 5	40%	168.00	336.00	504.00
7	Above 5	40%	196.00	392.00	588.00

5.5 Receipt generation

The method `generateReceipt()` builds a text receipt containing the vehicle number, vehicle type, entry time, exit time, parking hours and the parking fee formatted to two decimal places. The console version prints the receipt in the terminal and the GUI version shows it in a text area.

6. Methods used

All the logic is placed in methods, so the same code is used by both modes.

Method	Purpose	Returns
<code>calculateDuration()</code>	Finds the parking time in minutes from entry and exit time	long (-1 if time is invalid)
<code>getParkingHours()</code>	Rounds the minutes up to whole hours	int
<code>calculateFee()</code>	Selects the rate (switch) and applies the slab (if-else)	double
<code>generateReceipt()</code>	Builds the receipt text	String
<code>consoleMode()</code>	Runs the Scanner-based version	void
<code>createGUI()</code>	Builds the Swing window	void
<code>main()</code>	Asks for Console or GUI mode and starts it	void

7. Java concepts used

Concept	How it is used in this project
Scanner	<code>static Scanner scanner = new Scanner(System.in);</code> reads the mode, vehicle number, vehicle choice and the entry and exit times in console mode.
Switch-case	Used in <code>calculateFee()</code> to select the hourly rate for Bike, Car or Truck, and in <code>consoleMode()</code> to convert the menu number into a vehicle type.

Concept	How it is used in this project
Operators	Arithmetic operators (+, *, /) calculate the fee, the surcharge and the number of hours. Comparison operators (<, <=) check the duration and the slabs. The += operator builds the receipt.
If-else statements	Apply the three duration slabs, validate empty input, and check whether the calculated duration is valid.
Methods	calculateDuration(), getParkingHours(), calculateFee() and generateReceipt() split the work into small reusable parts.
Exception handling	try-catch handles DateTimeParseException for a wrong time format and NumberFormatException for a non-numeric menu choice.
Java Swing (extra)	JFrame, JLabel, JTextField, JComboBox, JButton, JTextArea and JOptionPane build the graphical interface.
java.time API	LocalTime, Duration and DateTimeFormatter read the times and find the difference.

8. Algorithm

1. Start the program.
2. Ask the user to choose Console Mode or GUI Mode.
3. Take the vehicle number and check that it is not empty.
4. Take the vehicle type (Bike, Car or Truck).
5. Take the entry time and the exit time in HH:mm format.
6. Calculate the parking duration in minutes. If it is negative, add 24 hours (parking after midnight).
7. If the duration is zero or the time is invalid, show an error and stop.
8. Round the minutes up to whole hours.
9. Select the hourly rate for the vehicle type using switch.
10. If hours ≤ 2 , fee = rate $\times$ hours.
11. Else if hours ≤ 5 , fee = rate $\times$ hours plus 20% surcharge.
12. Otherwise, fee = rate $\times$ hours plus 40% surcharge.
13. Generate the parking receipt and display it.
14. End the program.

9. Output screenshots

The screenshots below show the program running. The same inputs are used in console mode and in GUI mode: vehicle number MH12AB1234, vehicle type Car, entry time 10:00 and exit time 14:00.

9.1 Console mode

The program is compiled with javac and run with java. Choosing option 1 starts console mode, which asks for the vehicle number, vehicle type, entry time and exit time and then prints the receipt.

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
▼ TERMINAL  zsh
rashmeetkaur@Rashmeets-MacBook-Air JAVA PROJECT % javac ParkingFeeCalculator.java
rashmeetkaur@Rashmeets-MacBook-Air JAVA PROJECT % java ParkingFeeCalculator

=====
PARKING FEE CALCULATOR
=====
1. Console Mode
2. GUI Mode
Enter your choice: 1

=====
PARKING FEE CALCULATOR
=====
Enter vehicle number: MH12AB1234

Select Vehicle Type:
1. Bike
2. Car
3. Truck
Enter your choice: 2
Enter entry time (HH:mm): 10:00
Enter exit time (HH:mm): 14:00

=====
PARKING RECEIPT
=====
Vehicle Number : MH12AB1234
Vehicle Type   : Car
Entry Time     : 10:00
Exit Time      : 14:00
Parking Hours   : 4
Parking Fee    : ₹192.00
=====
Thank You!
=====
```

Figure 1: Console mode with the parking receipt

9.2 GUI mode

Choosing option 2 in the start menu opens the graphical version of the calculator.

```
=====
rashmeetkaur@Rashmeets-MacBook-Air JAVA PROJECT % java ParkingFeeCalculator
=====
PARKING FEE CALCULATOR
=====
1. Console Mode
2. GUI Mode
Enter your choice: 2
rashmeetkaur@Rashmeets-MacBook-Air JAVA PROJECT %
```

Figure 2: Selecting GUI mode

The screenshot shows a window titled "Parking Fee Calculator". Inside, the title "PARKING FEE CALCULATOR" is centered at the top. Below it are four input fields: "Vehicle Number:" with the value "MH12AB1234", "Vehicle Type:" with a dropdown menu showing "Car", "Entry Time (HH:mm):" with the value "10:00", and "Exit Time (HH:mm):" with the value "14:00". Below these fields is a "Calculate Fee" button. Underneath the button is a large, empty rectangular box for the receipt. At the bottom of the window is a "Clear" button.

Figure 3: GUI window with the inputs entered

After the Calculate Fee button is pressed, the receipt appears in the text area. The receipt area has a scroll bar, so Figure 5 shows the lower part of the same receipt.

This screenshot shows the same "Parking Fee Calculator" window, but now the "Calculate Fee" button has been pressed. The large rectangular box now contains a receipt. The receipt is titled "PARKING RECEIPT" and lists the following details: "Vehicle Number : MH12AB1234", "Vehicle Type : Car", "Entry Time : 10:00", "Exit Time : 14:00", "Parking Hours : 4", and "Parking Fee : ₹192.00". The receipt is enclosed in a box with a vertical scrollbar on the right side.

Figure 4: GUI showing the parking receipt

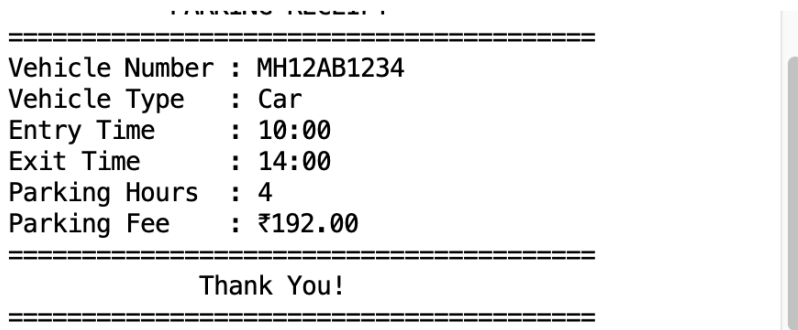


Figure 5: Lower part of the receipt (scrolled)

10. Validation and error handling

The program checks the input so that wrong values do not produce a wrong fee. If the user leaves the vehicle number empty in the GUI, a message box asks for it, as shown below.

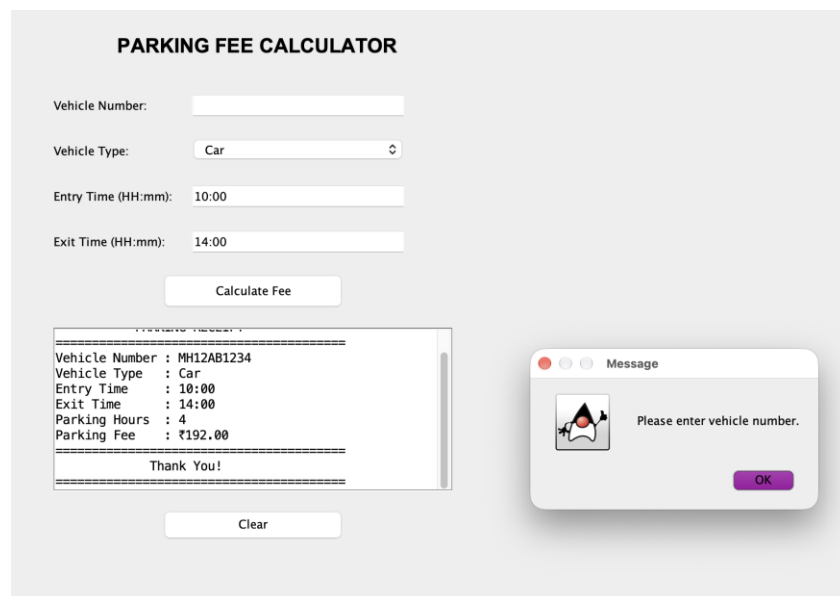


Figure 6: Validation message for an empty vehicle number

The other checks are listed below.

Situation	What the program does
Vehicle number is empty	Console: "Vehicle number cannot be empty." GUI: "Please enter vehicle number."
Entry or exit time is empty (GUI)	Displays "Please enter entry time." or "Please enter exit time."
Vehicle choice is not 1, 2 or 3	Displays "Invalid vehicle choice." and stops
A letter is typed for the vehicle choice	NumberFormatException is caught and "Invalid vehicle choice." is displayed

Situation	What the program does
Time is not in HH:mm format (for example 25:00)	calculateDuration() returns -1 and an invalid time message is displayed
Entry time and exit time are the same	Duration is 0 minutes, so an invalid time message is displayed
Exit time is earlier than entry time	Treated as parking after midnight; 24 hours are added
Start menu choice other than 1 or 2	Displays "Invalid choice."

11. Testing

The program was tested with different inputs and the displayed result was compared with the expected result. Test case 2 matches the screenshots in Section 9. The expected fees in the other cases are worked out from the rates and slabs in Section 5.4.

No.	Test case	Input	Expected result	Result
1	Bike, up to 2 hours	Bike, 09:00 to 10:30	2 hours, Rs. 40.00	Pass
2	Car, 3 to 5 hours	Car, 10:00 to 14:00	4 hours, Rs. 192.00	Pass
3	Truck, more than 5 hours	Truck, 08:00 to 15:00	7 hours, Rs. 588.00	Pass
4	Part of an hour	Car, 10:05 to 10:35	1 hour, Rs. 40.00	Pass
5	Rounding up	Bike, 10:00 to 13:01	4 hours, Rs. 96.00	Pass
6	Boundary: 2 hours	Car, 10:00 to 12:00	2 hours, Rs. 80.00	Pass
7	Boundary: 5 hours	Car, 10:00 to 15:00	5 hours, Rs. 240.00	Pass
8	Boundary: 6 hours	Car, 10:00 to 16:00	6 hours, Rs. 336.00	Pass
9	Parking after midnight	Car, 22:00 to 01:00	3 hours, Rs. 144.00	Pass
10	Invalid time format	Entry 25:00	Error message	Pass
11	Same entry and exit time	10:00 and 10:00	Error message	Pass
12	Empty vehicle number	Blank field in GUI	Message box	Pass
13	Wrong vehicle choice	Choice 5 or abc	Invalid vehicle choice	Pass

12. Future enhancements

The present version is a working prototype that demonstrates the core Java concepts. It can be extended into a production-ready parking system in the following ways:

- **Database integration:** store every entry, exit and receipt in a database such as MySQL or PostgreSQL, so that parking history and daily revenue reports are available.

- **Date and time support:** accept the date along with the time (using `LocalDateTime`) so that parking for more than 24 hours can be charged correctly.
- **Configurable pricing:** move the hourly rates and slab percentages to a configuration file or admin screen instead of fixing them in the code.
- **Passes and discounts:** add monthly passes, special-hour discounts and a penalty for a lost ticket.
- **Automatic timing:** record the entry time automatically from the system clock and calculate the exit time at the gate, removing manual typing errors.
- **Receipt export:** save or print the receipt as a PDF, and add a unique receipt number and date.
- **Digital payments:** integrate UPI, card or wallet payment before the exit barrier opens.
- **Number plate recognition:** read the vehicle number from a camera image to speed up entry and exit.
- **Slot management:** show the number of free slots and guide the driver to an available space.
- **Better design:** use classes (for example `Vehicle` and `Receipt`) and unit tests to make the code easier to maintain.

13. Conclusion

The Parking Fee Calculator meets every requirement of the case study. It accepts vehicle information, calculates the parking duration, applies vehicle-specific rates, applies duration-based slabs and generates a clear parking receipt. The program was tested with normal inputs, boundary values (2, 5 and 6 hours), parking after midnight and invalid inputs, and the results matched the expected fees.

The project brought together the main Java topics from the course. `Scanner` is used to accept input, a switch statement selects the rate for each vehicle type, operators perform the fee and surcharge calculation, if-else statements apply the duration slabs, and methods keep the logic reusable and organised. The same methods serve both the console version and the Swing interface, which showed me the advantage of separating logic from input and output.

Key learning outcomes

- Using the `java.time` API (`LocalTime` and `Duration`) to calculate time differences, including parking that crosses midnight.
- Rounding up with `Math.ceil()` so that any part of an hour is charged as a full hour.
- Validating user input and handling exceptions so that wrong data does not produce a wrong fee.
- Designing a modular program in which each method has one clear responsibility.
- Building a simple graphical interface with Java Swing alongside a console version.

Limitations

The calculator works on the time of day only, so it cannot charge parking longer than 24 hours, and it does not store records between runs. These limitations are addressed in the future enhancements above.

Overall, the project gave me practical experience in turning a real-world problem into a working Java program, and it provides a solid base for building a complete parking management system.

14. Source Code

GitHub Link- <https://github.com/krashmeet033-sys/JAVA-FINAL-PROJECT/tree/main>

The core methods of ParkingFeeCalculator.java are given below.

Duration and hours

```
public static long calculateDuration(String entryTime, String exitTime) {
    try {
        LocalTime entry = LocalTime.parse(entryTime, timeFormatter);
        LocalTime exit = LocalTime.parse(exitTime, timeFormatter);
        long minutes = Duration.between(entry, exit).toMinutes();
        // Handles parking after midnight
        if (minutes < 0) {
            minutes += 24 * 60;
        }
        return minutes;
    } catch (DateTimeParseException e) {
        return -1;
    }
}

public static int getParkingHours(long minutes) {
    return (int) Math.ceil(minutes / 60.0);
}
```

Fee calculation

```
public static double calculateFee(String vehicleType, int hours) {
    double hourlyRate = 0;
    double fee;
    switch (vehicleType) {
        case "Bike": hourlyRate = 20; break;
        case "Car": hourlyRate = 40; break;
        case "Truck": hourlyRate = 60; break;
        default:
            System.out.println("Invalid vehicle type.");
            return 0;
    }
    if (hours <= 2) {
        fee = hourlyRate * hours;
    } else if (hours <= 5) {
        fee = (hourlyRate * hours) + (hourlyRate * hours * 0.20);
    } else {
        fee = (hourlyRate * hours) + (hourlyRate * hours * 0.40);
    }
    return fee;
}
```

Receipt generation

```
public static String generateReceipt(String vehicleNumber, String vehicleType,
    String entryTime, String exitTime, int hours, double fee) {
    String receipt = "";
    receipt += "=====\n";
    receipt += "                PARKING RECEIPT\n";
    receipt += "=====\n";
    receipt += "Vehicle Number : " + vehicleNumber + "\n";
    receipt += "Vehicle Type   : " + vehicleType + "\n";
    receipt += "Entry Time     : " + entryTime + "\n";
    receipt += "Exit Time      : " + exitTime + "\n";
    receipt += "Parking Hours  : " + hours + "\n";
    receipt += "Parking Fee    : ₹" + String.format("%.2f", fee) + "\n";
    receipt += "=====\n";
}
```

```
receipt += "                Thank You!\n";  
receipt += "=====";  
return receipt;  
}
```

The remaining methods, `consoleMode()` and `createGUI()`, read the inputs with Scanner or Swing components, validate them, and call the methods above. The complete source code is available at the [GitHub link](#) given above.